

Структуры в C++

- **Структура** - это совокупность переменных, объединенных одним именем, предоставляющая общепринятый способ совместного хранения информации.
- Объявление структуры приводит к образованию шаблона, используемого для создания объектов структуры.
- Переменные, образующие структуру, называются **членами структуры**. (Члены структуры также часто называются **элементами** или **полями**.)
- Структура — это агрегатный тип данных, так как может содержать в себе разнотипные элементы.

Описание структуры

```
struct <имя структуры>
{
    <тип данных элемента 1 структуры> имя1;
    <тип данных элемента 2 структуры> имя2;
    ...
};
```

Пример

```
struct PlayerInfo           // сведения об игроке
{
    string name;             // имя
    int age;                  // возраст
    int skill_level;         // уровень
    bool isplaying;          // статус: в игре/не в игре
};
```

- Так как структура это тип данных, то, для того, чтобы использовать этот тип, необходимо объявить структурную переменную, а вместо типа данных указать имя структуры.

`<имя структуры> <имя переменной>;`

Пример:

`PlayerInfo CrazyMax;`

- Когда объявлена структурная переменная, компилятор автоматически выделяет необходимый участок памяти для размещения всех ее членов.

Доступ к членам структуры

- Доступ к отдельным членам структуры осуществляется с помощью оператора . (точка)

имя_структуры.имя_члена

Пример:

```
CrazyMax.name = "Максим Иванов";
```

```
CrazyMax.age = 15;
```

```
CrazyMax.skill_level = 100;
```

```
CrazyMax.isplaying = true;
```

Присваивание структур

- Информация, содержащаяся в одной структуре, может быть присвоена другой структуре того же типа с помощью одиночного оператора присваивания, то есть не нужно присваивать значение каждого члена по отдельности.
- Каждая вновь создаваемая структурная переменная содержит свои собственные копии переменных, образующих структуру.

Пример:

```
PlayerInfo CurrentPlayer;  
CurrentPlayer = CrazyMax;
```

Массивы структур

- Так же как и с простыми типами, можно создавать массивы структур. Тогда с каждым элементом этого массива работать так же как и с отдельной переменной.
- Для объявления массива структур **следует сначала определить структуру**, а затем объявить массив переменных данного типа.
- Как обычно, массивы структур индексируются с нуля.

Описание массива структур

```
struct PlayerInfo
{
    string name;
    int age;
    int skill_level;
    bool isplaying;
};

PlayerInfo players[5];
```

Доступ к элементам указателя на структуру

- Для получения адреса структурной переменной следует поместить оператор & перед именем структуры.
- Для доступа к членам структуры с помощью указателя на структуру следует использовать оператор "стрелка". Оператор "стрелка", ->, образован из знака "минус" и символа "больше".

Пример обращения к членам структуры через указатель

```
struct PlayerInfo
{
    string name;
    int age;
    int skill_level;
} CrazyMax;
struct PlayerInfo *ptr;
ptr = &CrazyMax;
ptr->age = 16;
```

Структуры и функции

- Очень часто требуется описать функцию, которая принимает структуры в качестве аргумента или возвращает структуру.

```
void update(struct my_time *t);
```

// в функцию передается указатель на структуру

Функция возвращает структуру

```
struct EnemySpaceShip {  
    int x_coordinate;  
    int y_coordinate;  
    int weapon_power;  
};  
  
EnemySpaceShip upgradeWeapons (EnemySpaceShip  
ship)  
{  
    ship.weapon_power += 10;  
    return ship;  
}
```

Битовые поля

- Язык С имеет возможность, называемую **битовыми полями**, позволяющую работать с отдельными битами. Битовые поля полезны по нескольким причинам:
 1. Если ограничено место для хранения информации, можно сохранить несколько логических (истина/ложь) переменных в одном байте.
 2. Некоторые интерфейсы устройств передают информацию, закодировав биты в один байт.
 3. Некоторым процедурам кодирования необходимо получить доступ к отдельным битам в байте.

- Метод использования битовых полей для доступа к битам основан на структурах.
- Битовое поле, на самом деле, - это просто особый тип структуры, определяющей, какую длину имеет каждый член.
- Стандартный вид объявления битовых полей следующий:

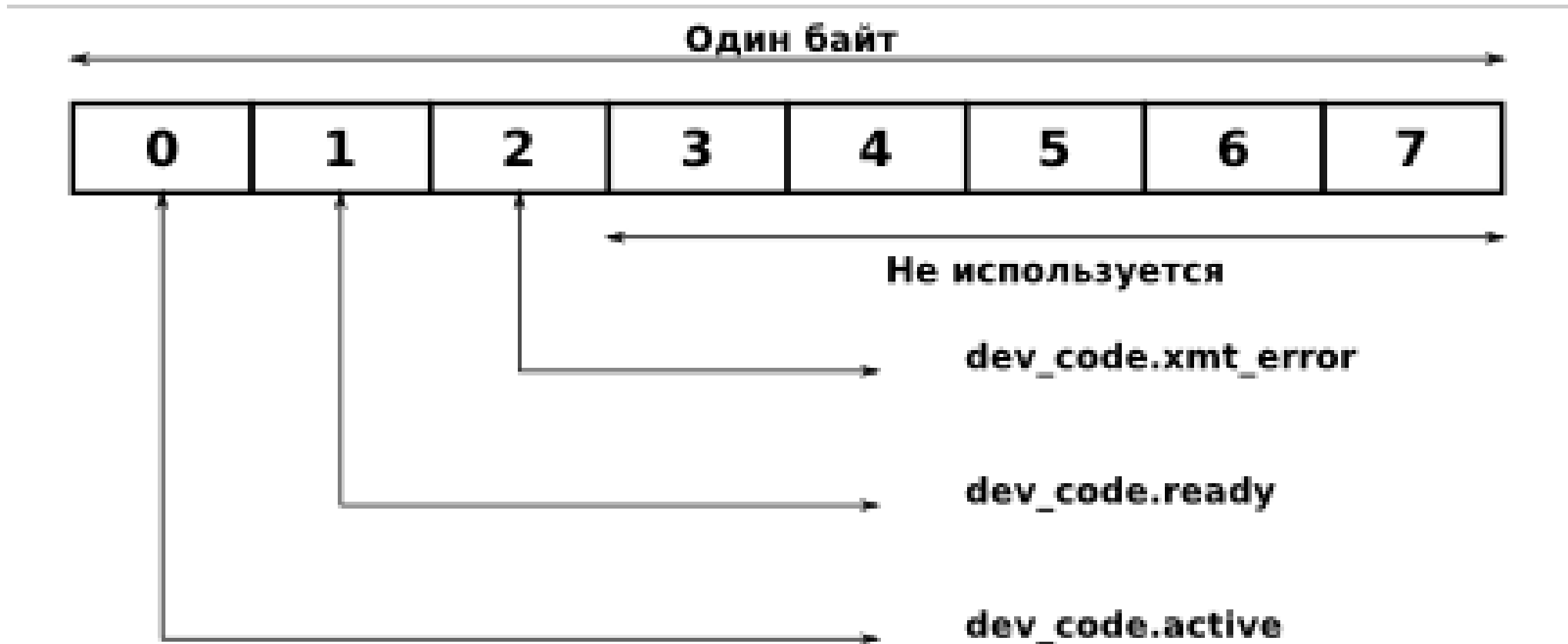
```
struct имя_структуры {  
тип имя1: длина;  
тип имя2: длина;  
...  
тип имяN: длина;  
}
```

- Битовые поля должны объявляться как `int`, `unsigned` или `signed`.
- Битовые поля длиной 1 должны объявляться как `unsigned`, поскольку 1 бит не может иметь знака.
- Битовые поля могут иметь длину от 1 до 16 бит для 16-битных сред и от 1 до 32 бит для 32-битных сред.
- Самый левый бит является знаковым.

Пример

- ```
struct device {
 unsigned active : 1;
 unsigned ready : 1;
 unsigned xmt_error : 1;
} dev_code;
```

# Размещение битового поля dev\_code в памяти



- Битовые поля имеют некоторые ограничения.
- Нельзя получить адрес переменной битового поля.
- Переменные битового поля не могут помещаться в массив.
- Переходя с одного компьютера на другой нельзя быть уверенным в порядке изменения битов (слева направо или справа налево). Любой код, использующий битовые поля, зависит от разрядности и архитектуры компьютера.

- Можно смешивать различные структурные переменные в битовых полях.

```
struct employee {
 struct addr address;
 float pay;
 unsigned lay_off:1;
 unsigned hourly:1;
 unsigned deductions:3;
};
```